

Ejemplos resueltos de atención en Transformers

Apunte de apoyo didáctico

Propósito del documento

Este material resume, con ejemplos numéricos pequeños, los conceptos que suelen costar más al estudiar Transformers: proyecciones Q , K , V , $softmax$, $self-attention$, matriz de atención y $multi-head attention$. El objetivo no es reproducir un modelo real a gran escala, sino mostrar con números sencillos qué hace cada bloque y por qué es importante.

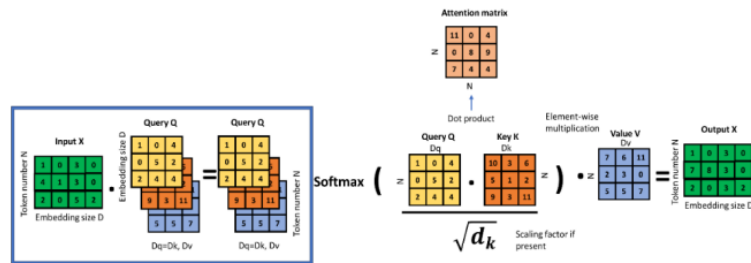
1. Idea general

Un Transformer toma una secuencia de tokens, convierte cada token en un vector y luego permite que cada token **mire a otros tokens de la misma secuencia** para construir una nueva representación contextualizada. La operación central es:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \quad (1)$$

La lectura conceptual es la siguiente:

- Q (*query*): qué está buscando cada token.
- K (*key*): cómo puede ser encontrado cada token.
- V (*value*): qué información aporta cada token si recibe atención.
- QK^T : puntajes de similitud entre tokens.
- $softmax$: convierte los puntajes en pesos que suman 1.
- Multiplicar por V : combina la información relevante y produce una salida contextualizada.



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Figura 1: Visión global de la atención con producto punto escalado.

2. Cómo leer las figuras de apoyo

La primera figura resume el flujo completo. A la izquierda está la matriz de entrada X . A partir de ella se obtienen Q , K y V mediante proyecciones lineales:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V. \quad (2)$$

Luego, cada fila de Q se compara con todas las filas de K , generando una **matriz de atención**. Después, *softmax* transforma esos puntajes en pesos, y esos pesos se usan para combinar las filas de V . El resultado final es una nueva matriz de salida donde cada token ya no representa solo su identidad léxica, sino también el contexto de la oración.

El diagrama ilustra el funcionamiento de la función softmax. En la parte superior, se define la fórmula general: $y = \frac{e^{x_i}}{\sum_{i=1}^n e^{x_i}}$. Abajo, se muestra un ejemplo numérico. A la izquierda, se define $y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$. En el centro, se indica que $y = \text{softmax}(x) = f\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right)$. A la derecha, se muestra la matriz resultante de la transformación: $\begin{bmatrix} \frac{e^{x_1}}{e^{x_1} + e^{x_2} + e^{x_3}} \\ \frac{e^{x_2}}{e^{x_1} + e^{x_2} + e^{x_3}} \\ \frac{e^{x_3}}{e^{x_1} + e^{x_2} + e^{x_3}} \end{bmatrix}$. En la parte inferior, se introduce el ejemplo de Python: $\text{python : } y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \text{softmax}(x) = f\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \frac{\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}}{e^{x_1} + e^{x_2} + e^{x_3}} = \frac{e^x}{\text{sum}(e^x)}$.

Figura 2: *Softmax* convierte puntajes en una distribución de pesos.

3. Ejemplo 1: atención completa sobre una oración corta

Usaremos la oración:

El gato come pescado

La idea del ejemplo es mostrar, **token por token**, cómo se forma la matriz de atención y cómo se obtiene la salida final.

3.1. Paso 1: matriz de entrada X

Asignamos embeddings muy pequeños, de dimensión 2:

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 2 \\ 1 & 1 \end{bmatrix} \quad (3)$$

Interpretación de filas:

$$\begin{aligned} t_1 &= \text{El} = [1, 0], \\ t_2 &= \text{gato} = [1, 1], \\ t_3 &= \text{come} = [0, 2], \\ t_4 &= \text{pescado} = [1, 1]. \end{aligned}$$

Para no cargar el ejemplo con demasiadas cuentas, tomaremos el caso más simple:

$$W^Q = W^K = W^V = I. \quad (4)$$

Por tanto,

$$Q = K = V = X. \quad (5)$$

3.2. Paso 2: construir $QK^\top$

Calculamos el producto punto de cada token contra todos los demás:

$$QK^\top = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 1 & 2 & 2 & 2 \\ 0 & 2 & 4 & 2 \\ 1 & 2 & 2 & 2 \end{bmatrix}. \quad (6)$$

Cada fila responde a la pregunta: *¿a qué tokens se parece la query del token actual?*

Por ejemplo, la tercera fila corresponde a **come**. Sus puntajes son:

$$[0, 2, 4, 2]. \quad (7)$$

Eso indica que, con estos embeddings, **come** se parece mucho más a sí mismo que a los demás tokens.

3.3. Paso 3: escalar por $\sqrt{d_k}$

Como $d_k = 2$, entonces $\sqrt{d_k} = \sqrt{2} \approx 1.414$. La matriz escalada es:

$$\frac{QK^\top}{\sqrt{2}} = \begin{bmatrix} 0.707 & 0.707 & 0 & 0.707 \\ 0.707 & 1.414 & 1.414 & 1.414 \\ 0 & 1.414 & 2.828 & 1.414 \\ 0.707 & 1.414 & 1.414 & 1.414 \end{bmatrix}. \quad (8)$$

El escalado es importante porque evita que los productos punto crezcan demasiado y saturen el *softmax*.

3.4. Paso 4: aplicar *softmax* fila por fila

La matriz de atención queda:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \approx \begin{bmatrix} 0.286 & 0.286 & 0.141 & 0.286 \\ 0.141 & 0.286 & 0.286 & 0.286 \\ 0.038 & 0.157 & 0.647 & 0.157 \\ 0.141 & 0.286 & 0.286 & 0.286 \end{bmatrix}. \quad (9)$$

Ahora sí tenemos una lectura probabilística. Por ejemplo:

- El atiende casi por igual a **El**, **gato** y **pescado**, y menos a **come**.
- **come** pone el mayor peso en sí mismo (0.647), pero también incorpora algo de **gato** y **pescado**.

3.5. Paso 5: multiplicar por V

La salida es:

$$O = AV. \quad (10)$$

Al hacer la multiplicación obtenemos:

$$O \approx \begin{bmatrix} 0.859 & 0.855 \\ 0.714 & 1.145 \\ 0.353 & 1.609 \\ 0.714 & 1.145 \end{bmatrix}. \quad (11)$$

3.6. Interpretación token por token

- **El**: su salida ya no es $[1, 0]$, sino $[0.859, 0.855]$. Esto muestra que el artículo incorpora contexto de toda la secuencia.
- **gato**: su nuevo vector es $[0.714, 1.145]$. Recoge información de sí mismo, del verbo y del objeto.
- **come**: termina como $[0.353, 1.609]$, claramente influido por el resto de la oración.
- **pescado**: queda igual que **gato** en este ejemplo porque ambos tenían el mismo embedding inicial.

La lección importante es que **cada token cambia de representación al mirar al resto**. Ese es el sentido profundo de la figura de atención: el embedding de salida no representa solo el token aislado, sino el token *en contexto*.

4. Ejemplo 2: lectura token a token de una oración ambigua

Una gran ventaja de la atención es resolver ambigüedad léxica. Pensemos en la palabra **banco**.

Oración A

banco del parque

Supongamos que para el token **banco** los puntajes son:

$$[\text{banco} = 34, \text{del} = 13, \text{parque} = 53]. \quad (12)$$

Después de *softmax*, podríamos tener algo conceptual como:

$$[0.4, 0.0, 0.6]. \quad (13)$$

Esto quiere decir que **banco** atiende sobre todo a **parque**. Por tanto, su vector de salida se acerca al significado de *asiento*.

Oración B

banco y dinero

Si ahora los puntajes del token **banco** fueran más altos hacia **dinero**, la salida final se movería hacia el sentido de *institución financiera*. El token es el mismo; lo que cambia es **a qué contexto atiende**.

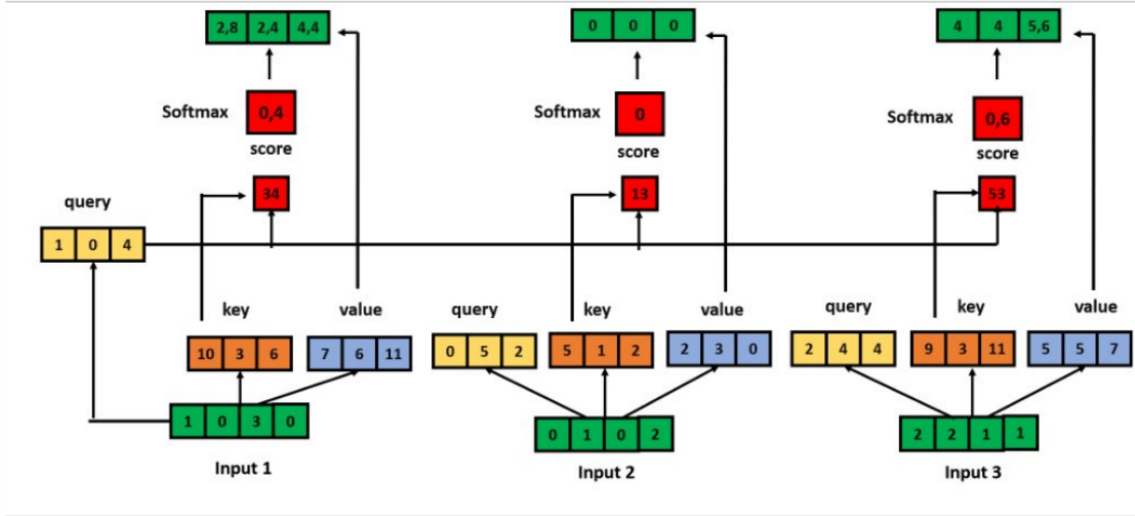


Figura 3: Lectura token a token: una query compara con todas las keys, genera pesos y combina values.

5. Ejemplo 3: *multi-head attention* con números pequeños

Ahora haremos un ejemplo completo, cabecera por cabecera. La figura original habla de 8 cabeceras; aquí usaremos 2 para poder calcular todo a mano.

Secuencia de tokens:

perro ladra fuerte

Tomamos un embedding total de dimensión 4:

$$X = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \\ 1 & 1 & 1 & 1 \end{bmatrix}. \quad (14)$$

5.1. Paso 1: dividir en cabeceras

Usaremos 2 cabeceras, así que cada una trabaja con dimensión 2.

Cabecera 1. Usa las dos primeras dimensiones:

$$X^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}. \quad (15)$$

Cabecera 2. Usa las dos últimas dimensiones:

$$X^{(2)} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}. \quad (16)$$

Otra vez, para fines didácticos, tomaremos:

$$Q^{(h)} = K^{(h)} = V^{(h)} = X^{(h)}. \quad (17)$$

5.2. Cabecera 1 completa

$$Q^{(1)}K^{(1)\top} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix} \implies \frac{Q^{(1)}K^{(1)\top}}{\sqrt{2}} = \begin{bmatrix} 0.707 & 0 & 0.707 \\ 0 & 0.707 & 0.707 \\ 0.707 & 0.707 & 1.414 \end{bmatrix}. \quad (18)$$

Aplicando *softmax* por filas:

$$A^{(1)} \approx \begin{bmatrix} 0.401 & 0.198 & 0.401 \\ 0.198 & 0.401 & 0.401 \\ 0.248 & 0.248 & 0.503 \end{bmatrix}. \quad (19)$$

La salida es:

$$O^{(1)} = A^{(1)}V^{(1)} \approx \begin{bmatrix} 0.802 & 0.599 \\ 0.599 & 0.802 \\ 0.751 & 0.751 \end{bmatrix}. \quad (20)$$

5.3. Cabecera 2 completa

$$Q^{(2)}K^{(2)\top} = \begin{bmatrix} 4 & 0 & 2 \\ 0 & 4 & 2 \\ 2 & 2 & 2 \end{bmatrix} \implies \frac{Q^{(2)}K^{(2)\top}}{\sqrt{2}} = \begin{bmatrix} 2.828 & 0 & 1.414 \\ 0 & 2.828 & 1.414 \\ 1.414 & 1.414 & 1.414 \end{bmatrix}. \quad (21)$$

Aplicando *softmax*:

$$A^{(2)} \approx \begin{bmatrix} 0.768 & 0.045 & 0.187 \\ 0.045 & 0.768 & 0.187 \\ 0.333 & 0.333 & 0.333 \end{bmatrix}. \quad (22)$$

La salida queda:

$$O^{(2)} = A^{(2)}V^{(2)} \approx \begin{bmatrix} 1.723 & 0.277 \\ 0.277 & 1.723 \\ 1.000 & 1.000 \end{bmatrix}. \quad (23)$$

5.4. Concatenación de cabeceras

Cada token ahora tiene dos salidas parciales, una por cabecera. Las concatenamos:

$$O_{\text{concat}} = \begin{bmatrix} 0.802 & 0.599 & 1.723 & 0.277 \\ 0.599 & 0.802 & 0.277 & 1.723 \\ 0.751 & 0.751 & 1.000 & 1.000 \end{bmatrix}. \quad (24)$$

Esta matriz es la que luego se proyecta con una matriz final W^O .

5.5. ¿Qué nos enseña este ejemplo?

Las dos cabeceras **no atienden igual**. Para el token 1:

- Cabecera 1: pesos $[0.401, 0.198, 0.401]$. Reparte atención entre sí misma y el token 3.

- Cabecera 2: pesos $[0.768, 0.045, 0.187]$. Se concentra más en sí misma.

Esto muestra por qué *multi-head attention* es importante: diferentes cabeceras pueden capturar relaciones diferentes al mismo tiempo. En un modelo real, una cabecera puede aprender estructura sintáctica, otra negación, otra coreferencia, otra relaciones semánticas, etcétera.

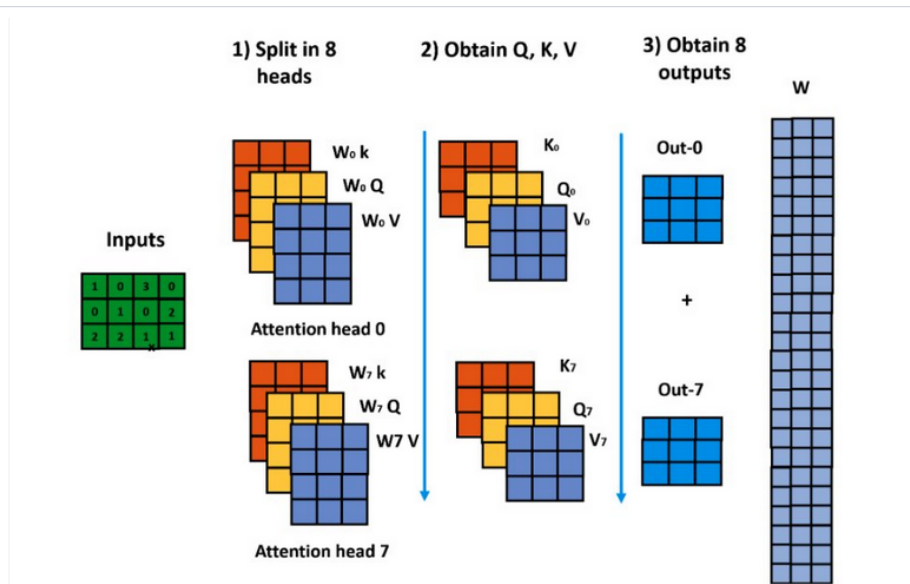


Figura 4: *Multi-head attention*: varias atenciones paralelas y posterior concatenación/proyección.

6. Cómo se conecta esto con el bloque Transformer completo

La atención no trabaja sola. En un bloque Transformer típico aparecen además:

- **Conexión residual:** suma la entrada original con la salida de atención.
- **LayerNorm:** estabiliza la escala de activaciones.
- **FFN:** red feed-forward que transforma cada token de forma no lineal.

Una forma típica de escribir el bloque es:

$$H = \text{LayerNorm}(X + \text{MultiHeadSelfAttention}(X)), \quad (25)$$

$$Y = \text{LayerNorm}(H + \text{FFN}(H)). \quad (26)$$

La intuición es esta:

- la atención mezcla información entre tokens;
- la conexión residual evita perder la señal original;
- LayerNorm estabiliza el entrenamiento;
- la FFN refina la representación de cada token después de haber incorporado contexto.

7. Checklist de estudio

Al terminar este material, un estudiante debería poder responder estas preguntas:

1. ¿Qué papel cumplen Q , K y V ?
2. ¿Por qué se calcula $QK^\top$?
3. ¿Por qué se divide entre $\sqrt{d_k}$?
4. ¿Qué hace *softmax* en la atención?
5. ¿Cómo se interpreta una fila de la matriz de atención?
6. ¿Qué significa que un token quede *contextualizado*?
7. ¿Por qué varias cabeceras son mejores que una sola?
8. ¿Cómo se combinan las salidas de distintas cabeceras?

Resumen final

La idea esencial del Transformer es simple de enunciar, aunque poderosa en la práctica: **cada token reconstruye su representación utilizando información relevante de otros tokens**. La atención hace visible *quién mira a quién*; *softmax* convierte eso en pesos; la multiplicación por V mezcla contenido; y *multi-head attention* permite hacerlo desde varias perspectivas a la vez. Con esto, un embedding deja de ser un vector fijo y pasa a representar el token **dentro de su contexto**.